JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.chrono

Interface ChronoLocalDate

All Superinterfaces:

`Comparable<ChronoLocalDate>`, `Temporal`, `TemporalAccessor`, `TemporalAdjuster`

All Known Implementing Classes:

`HijrahDate`, `JapaneseDate`, `LocalDate`, `MinguoDate`, `ThaiBuddhistDate`

```
public interface ChronoLocalDate
extends Temporal, TemporalAdjuster, Comparable<ChronoLocalDate>
```

A date without time-of-day or time-zone in an arbitrary chronology, intended for advanced globalization use cases.

Most applications should declare method signatures, fields and variables as `LocalDate`, not this interface.

A `ChronoLocalDate` is the abstract representation of a date where the `Chronology chronology`, or calendar system, is pluggable. The date is defined in terms of fields expressed by `TemporalField`,
where most common implementations are defined in `ChronoField`. The chronology defines how the calendar system operates and the meaning of the standard fields.

When to use this interface

The design of the API encourages the use of `LocalDate` rather than this interface, even in the case where the application needs to deal with multiple calendar systems.

This concept can seem surprising at first, as the natural way to globalize an application might initially appear to be to abstract the calendar system. However, as explored below, abstracting the calendar system is usually the wrong approach, resulting in logic errors and hard to find bugs. As such, it should be considered an

application-wide architectural decision to choose to use this interface as opposed to `LocalDate`.

Architectural issues to consider

These are some of the points that must be considered before using this interface throughout an application.

1) Applications using this interface, as opposed to using just `LocalDate`, face a significantly higher probability of bugs. This is because the calendar system in use is not known at development time. A key cause of bugs is where the developer applies assumptions from their day-to-day knowledge of the ISO calendar system to code that is intended to deal with any arbitrary calendar system. The section below outlines how those assumptions can cause problems The primary mechanism for reducing this increased risk of bugs is a strong code review process. This should also be considered a extra cost in maintenance for the lifetime of the code.

2) This interface does not enforce immutability of implementations. While the implementation notes indicate that all implementations must be immutable there is nothing in the code or type system to enforce this. Any method declared to accept a `ChronoLocalDate` could therefore be passed a poorly or maliciously written mutable implementation.

3) Applications using this interface must consider the impact of eras. `LocalDate` shields users from the concept of eras, by ensuring that `getYear()` returns the proleptic year. That decision ensures that developers can think of `LocalDate` instances as consisting of three fields - year, month-of-year and day-of-month. By contrast, users of this interface must think of dates as consisting of four fields - era, year-of-era, month-of-year and day-of-month. The extra era field is frequently forgotten, yet it is of vital importance to dates in an arbitrary calendar system. For example, in the Japanese calendar system, the era represents the reign of an Emperor. Whenever one reign ends and another starts, the year-of-era is reset to one.

4) The only agreed international standard for passing a date between two systems is the ISO-8601 standard which requires the ISO calendar system. Using this interface throughout the application will inevitably lead to the requirement to pass the date across a network or component boundary, requiring an application specific protocol or format.

5) Long term persistence, such as a database, will almost always only accept dates in the ISO-8601 calendar system (or the related Julian-Gregorian). Passing around dates in other calendar systems increases the complications of interacting with persistence.

6) Most of the time, passing a `ChronoLocalDate` throughout an application is unnecessary, as discussed in the last section below.

False assumptions causing bugs in multi-calendar system code

As indicated above, there are many issues to consider when try to use and manipulate a date in an arbitrary calendar system. These are some of the key issues.

Code that queries the day-of-month and assumes that the value will never be more than 31 is invalid. Some calendar systems have more than 31 days in some months.

Code that adds 12 months to a date and assumes that a year has been added is invalid. Some calendar systems have a different number of months, such as 13 in the Coptic or Ethiopic.

Code that adds one month to a date and assumes that the month-of-year value will increase by one or wrap to the next year is invalid. Some calendar systems have a variable number of months in a year, such as the Hebrew.

Code that adds one month, then adds a second one month and assumes that the day-of-month will remain close to its original value is invalid. Some calendar systems have a large difference between the length of the longest month and the length of the shortest month. For example, the Coptic or Ethiopic have 12 months of 30 days and 1 month of 5 days.

Code that adds seven days and assumes that a week has been added is invalid. Some calendar systems have weeks of other than seven days, such as the French Revolutionary.

Code that assumes that because the year of `date1` is greater than the year of `date2` then `date1` is after `date2` is invalid. This is invalid for all calendar systems when referring to the year-of-era, and especially untrue of the Japanese calendar system where the year-of-era restarts with the reign of every new Emperor.

Code that treats month-of-year one and day-of-month one as the start of the year is invalid. Not all calendar systems start the year when the month value is one.

In general, manipulating a date, and even querying a date, is wide open to bugs when the calendar system is unknown at development time. This is why it is essential that code using this interface is subjected to additional code reviews. It is also why an architectural decision to avoid this interface type is usually the correct one.

Using LocalDate instead

The primary alternative to using this interface throughout your application is as follows.

- Declare all method signatures referring to dates in terms of `LocalDate`.
- Either store the chronology (calendar system) in the user profile or lookup the chronology from the user locale
- Convert the ISO `LocalDate` to and from the user's preferred calendar system during printing and parsing

This approach treats the problem of globalized calendar systems as a localization issue and confines it to the UI layer. This approach is in keeping with other localization issues in the java platform.

As discussed above, performing calculations on a date where the rules of the calendar system are pluggable requires skill and is not recommended. Fortunately, the need to perform calculations on a date in an arbitrary calendar system is extremely rare. For example, it is highly unlikely that the business rules of a library book rental scheme will allow rentals to be for one month, where meaning of the month is dependent on the user's preferred calendar system.

A key use case for calculations on a date in an arbitrary calendar system is producing a month-by-month calendar for display and user interaction. Again, this is a UI issue, and use of this interface solely within a few methods of the UI layer may be justified.

In any other part of the system, where a date must be manipulated in a calendar system other than ISO, the use case will generally specify the calendar system to use. For example, an application may need to calculate the next Islamic or Hebrew holiday which may require manipulating the date. This kind of use case can be handled as follows:

- start from the ISO `LocalDate` being passed to the method
- convert the date to the alternate calendar system, which for this use case is known rather than arbitrary
- perform the calculation
- convert back to `LocalDate`

Developers writing low-level frameworks or libraries should also avoid this interface. Instead, one of the two general purpose access interfaces should be used. Use `TemporalAccessor` if read-only access is required, or use `Temporal` if read-write access is required.

Implementation Requirements:

`This interface must be implemented with care to ensure other classes operate correctly. All implementations that can be instantiated must be final, immutable and thread-safe. Subclasses should be Serializable wherever possible.`

`Additional calendar systems may be added to the system. See Chronology for more details.`

Since:

`1.8`

Method Summary

Modifier and Type	Method	Default M… Desc…

default **Temporal**	**adjustInto**(**Temporal** temporal)	Adjus temp have this c
default **ChronoLocalDateTime**<?>	**atTime**(**LocalTime** localTime)	Comb with a tim Chro
default int	**compareTo**(**ChronoLocalDate** other)	Comp anoth inclu chron
boolean	**equals**(**Object** obj)	Chec equa inclu chron
default **String**	**format**(**DateTimeFormatter** formatter)	Form using forma
static **ChronoLocalDate**	**from**(**TemporalAccessor** temporal)	Obta Chro a tem
Chronology	**getChronology**()	Gets this c
default **Era**	**getEra**()	Gets defin chron
int	**hashCode**()	A has date.
default boolean	**isAfter**(**ChronoLocalDate** other)	Chec after date chron
default boolean	**isBefore**(**ChronoLocalDate** other)	Chec befor date chron
default boolean	**isEqual**(**ChronoLocalDate** other)	

		Chec equa date chron
default boolean	**isLeapYear**()	Chec leap by th syste
default boolean	**isSupported**(**TemporalField** field)	Chec field
default boolean	**isSupported**(**TemporalUnit** unit)	Chec unit i
int	**lengthOfMonth**()	Retu the m by th defin calen
default int	**lengthOfYear**()	Retu the y by th defin calen
default **ChronoLocalDate**	**minus**(long amountToSubtract, **TemporalUnit** unit)	Retu the s objec speci subtr
default **ChronoLocalDate**	**minus**(**TemporalAmount** amount)	Retu the s objec subtr
default **ChronoLocalDate**	**plus**(long amountToAdd, **TemporalUnit** unit)	Retu the s objec speci adde
default **ChronoLocalDate**	**plus**(**TemporalAmount** amount)	Retu the s objec adde

default <R> R	**query**(**TemporalQuery**<R> query)	Quer using query
static **Comparator**<**ChronoLocalDate**>	**timeLineOrder**()	Gets that Chro time- ignor chron
default long	**toEpochDay**()	Conv the E
String	**toString**()	Outp Strin
ChronoPeriod	**until**(**ChronoLocalDate** endDateExclusive)	Calcu betw anoth Chro
long	**until**(**Temporal** endExclusive, **TemporalUnit** unit)	Calcu of tin date speci
default **ChronoLocalDate**	**with**(**TemporalAdjuster** adjuster)	Retu objec type with made
default **ChronoLocalDate**	**with**(**TemporalField** field, long newValue)	Retu the s objec speci

Methods inherited from interface java.time.temporal.TemporalAccessor

get, getLong, range

Method Detail

timeLineOrder

`static Comparator<ChronoLocalDate> timeLineOrder()`

Gets a comparator that compares `ChronoLocalDate` in time-line order ignoring the chronology.

This comparator differs from the comparison in `compareTo(java.time.chrono.ChronoLocalDate)` in that it only compares the underlying date and not the chronology. This allows dates in different calendar systems to be compared based on the position of the date on the local time-line. The underlying comparison is equivalent to comparing the epoch-day.

Returns:

`a comparator that compares in time-line order ignoring the chronology`

See Also:

`isAfter(java.time.chrono.ChronoLocalDate)`, `isBefore(java.time.chrono.ChronoLocalDate)`, `isEqual(java.time.chrono.ChronoLocalDate)`

from

`static ChronoLocalDate from(TemporalAccessor temporal)`

Obtains an instance of `ChronoLocalDate` from a temporal object.

This obtains a local date based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ChronoLocalDate`.

The conversion extracts and combines the chronology and the date from the temporal object. The behavior is equivalent to using `Chronology.date(TemporalAccessor)` with the extracted chronology. Implementations are permitted to perform optimizations such as accessing those fields that are equivalent to the relevant objects.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `ChronoLocalDate::from`.

Parameters:

`temporal - the temporal object to convert, not null`

Returns:

`the date, not null`

Throws:

`DateTimeException` - if unable to convert to a ChronoLocalDate

See Also:

`Chronology.date(TemporalAccessor)`

getChronology

`Chronology getChronology()`

Gets the chronology of this date.

The `Chronology` represents the calendar system in use. The era and other fields in `ChronoField` are defined by the chronology.

Returns:

the chronology, not null

getEra

`default Era getEra()`

Gets the era, as defined by the chronology.

The era is, conceptually, the largest division of the time-line. Most calendar systems have a single epoch dividing the time-line into two eras. However, some have multiple eras, such as one for the reign of each leader. The exact meaning is determined by the `Chronology`.

All correctly implemented `Era` classes are singletons, thus it is valid code to write `date.getEra() == SomeChrono.ERA_NAME)`.

This default implementation uses `Chronology.eraOf(int)`.

Returns:

the chronology specific era constant applicable at this date, not null

isLeapYear

`default boolean isLeapYear()`

Checks if the year is a leap year, as defined by the calendar system.

A leap-year is a year of a longer length than normal. The exact meaning is determined by the chronology with the constraint that a leap-year must imply a year-length longer than a non leap-year.

This default implementation uses `Chronology.isLeapYear(long)`.

Returns:

`true if this date is in a leap year, false otherwise`

lengthOfMonth

```
int lengthOfMonth()
```

Returns the length of the month represented by this date, as defined by the calendar system.

This returns the length of the month in days.

Returns:

`the length of the month in days`

lengthOfYear

```
default int lengthOfYear()
```

Returns the length of the year represented by this date, as defined by the calendar system.

This returns the length of the year in days.

The default implementation uses `isLeapYear()` and returns 365 or 366.

Returns:

`the length of the year in days`

isSupported

```
default boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if the specified field can be queried on this date. If false, then calling the `range`, `get` and `with(TemporalField, long)` methods will throw an exception.

The set of supported fields is defined by the chronology and normally includes all `ChronoField` date fields.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field - the field to check, null returns false`

Returns:

`true if the field can be queried, false if not`

isSupported

`default boolean isSupported(TemporalUnit unit)`

Checks if the specified unit is supported.

This checks if the specified unit can be added to or subtracted from this date. If false, then calling the `plus(long, TemporalUnit)` and `minus` methods will throw an exception.

The set of supported units is defined by the chronology and normally includes all `ChronoUnit` date units except `FOREVER`.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.isSupportedBy(Temporal)` passing `this` as the argument. Whether the unit is supported is determined by the unit.

Specified by:

`isSupported` in interface `Temporal`

Parameters:

`unit - the unit to check, null returns false`

Returns:

`true if the unit can be added/subtracted, false if not`

with

`default ChronoLocalDate with(TemporalAdjuster adjuster)`

Returns an adjusted object of the same type as this object with the adjustment made.

This adjusts this date-time according to the rules of the specified adjuster. A simple adjuster might simply set the one of the fields, such as the year field. A more complex adjuster might set the date to the last day of the month. A selection of common adjustments is provided in `TemporalAdjusters`. These include finding the "last day of the month" and "next Wednesday". The adjuster is responsible for handling special cases, such as the varying lengths of month and leap years.

Some example code indicating how and why this method is used:

```
  date = date.with(Month.JULY);        // most key classes implement TemporalAdju
  date = date.with(lastDayOfMonth());  // static import from Adjusters
  date = date.with(next(WEDNESDAY));   // static import from Adjusters and DayOfW
```

Specified by:

`with` in interface `Temporal`

Parameters:

`adjuster` - the adjuster to use, not null

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

with

```
default ChronoLocalDate with(TemporalField field,
                             long newValue)
```

Returns an object of the same type as this object with the specified field altered.

This returns a new object based on this one with the value for the specified field changed. For example, on a `LocalDate`, this could be used to set the year, month or day-of-month. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st January, then changing the month to February would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Specified by:

`with` in interface `Temporal`

Parameters:

`field` - the field to set in the result, not null

`newValue` - the new value of the field in the result

Returns:

an object of the same type with the specified field set, not null

Throws:

DateTimeException - if the field cannot be set

UnsupportedTemporalTypeException - if the field is not supported

ArithmeticException - if numeric overflow occurs

plus

```
default ChronoLocalDate plus(TemporalAmount amount)
```

Returns an object of the same type as this object with an amount added.

This adjusts this temporal, adding according to the rules of the specified amount. The amount is typically a Period but may be any other type implementing the TemporalAmount interface, such as Duration.

Some example code indicating how and why this method is used:

```
  date = date.plus(period);               // add a Period instance
  date = date.plus(duration);             // add a Duration instance
  date = date.plus(workingDays(6));       // example user-written workingDays me
```

Note that calling plus followed by minus is not guaranteed to return the same date-time.

Specified by:

plus in interface Temporal

Parameters:

amount - the amount to add, not null

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

DateTimeException - if the addition cannot be made

ArithmeticException - if numeric overflow occurs

plus

```
default ChronoLocalDate plus(long amountToAdd,
                             TemporalUnit unit)
```

Returns an object of the same type as this object with the specified period added.

This method returns a new object based on this one with the specified period added. For example, on a LocalDate, this could be used to add a number of

years, months or days. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st January, then adding one month would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd - the amount of the specified unit to add, may be negative`

`unit - the unit of the amount to add, not null`

Returns:

`an object of the same type with the specified period added, not null`

Throws:

`DateTimeException - if the unit cannot be added`

`ArithmeticException - if numeric overflow occurs`

minus

```
default ChronoLocalDate minus(TemporalAmount amount)
```

Returns an object of the same type as this object with an amount subtracted.

This adjusts this temporal, subtracting according to the rules of the specified amount. The amount is typically a `Period` but may be any other type implementing the `TemporalAmount` interface, such as `Duration`.

Some example code indicating how and why this method is used:

```
  date = date.minus(period);               // subtract a Period instance
  date = date.minus(duration);             // subtract a Duration instance
  date = date.minus(workingDays(6));       // example user-written workingDays me
```

Note that calling `plus` followed by `minus` is not guaranteed to return the same date-time.

Specified by:

`minus` in interface `Temporal`

Parameters:

`amount - the amount to subtract, not null`

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

DateTimeException - if the subtraction cannot be made

ArithmeticException - if numeric overflow occurs

minus

```
default ChronoLocalDate minus(long amountToSubtract,
                              TemporalUnit unit)
```

Returns an object of the same type as this object with the specified period subtracted.

This method returns a new object based on this one with the specified period subtracted. For example, on a `LocalDate`, this could be used to subtract a number of years, months or days. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st March, then subtracting one month would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Specified by:

minus in interface Temporal

Parameters:

amountToSubtract - the amount of the specified unit to subtract, may be negative

unit - the unit of the amount to subtract, not null

Returns:

an object of the same type with the specified period subtracted, not null

Throws:

DateTimeException - if the unit cannot be subtracted

UnsupportedTemporalTypeException - if the unit is not supported

ArithmeticException - if numeric overflow occurs

query

`default <R> R query(TemporalQuery<R> query)`

Queries this date using the specified query.

This queries this date using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

`query` in interface `TemporalAccessor`

Type Parameters:

`R` - the type of the result

Parameters:

`query` - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

`DateTimeException` - if unable to query (defined by the query)

`ArithmeticException` - if numeric overflow occurs (defined by the query)

adjustInto

`default Temporal adjustInto(Temporal temporal)`

Adjusts the specified temporal object to have the same date as this object.

This returns a temporal object of the same observable type as the input with the date changed to be the same as this.

The adjustment is equivalent to using `Temporal.with(TemporalField, long)` passing `ChronoField.EPOCH_DAY` as the field.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.with(TemporalAdjuster)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisLocalDate.adjustInto(temporal);
```

```
temporal = temporal.with(thisLocalDate);
```

This instance is immutable and unaffected by this method call.

Specified by:

`adjustInto` in interface `TemporalAdjuster`

Parameters:

`temporal` - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

until

```
long until(Temporal endExclusive,
           TemporalUnit unit)
```

Calculates the amount of time until another date in terms of the specified unit.

This calculates the amount of time between two `ChronoLocalDate` objects in terms of a single `TemporalUnit`. The start and end points are `this` and the specified date. The result will be negative if the end is before the start. The `Temporal` passed to this method is converted to a `ChronoLocalDate` using `Chronology.date(TemporalAccessor)`. The calculation returns a whole number, representing the number of complete units between the two dates. For example, the amount in days between two dates can be calculated using `startDate.until(endDate, DAYS)`.

There are two equivalent ways of using this method. The first is to invoke this method. The second is to use `TemporalUnit.between(Temporal, Temporal)`:

```
// these two lines are equivalent
amount = start.until(end, MONTHS);
amount = MONTHS.between(start, end);
```

The choice should be made based on which makes the code more readable.

The calculation is implemented in this method for `ChronoUnit`. The units `DAYS`, `WEEKS`, `MONTHS`, `YEARS`, `DECADES`, `CENTURIES`, `MILLENNIA` and `ERAS` should be supported by all implementations. Other `ChronoUnit` values will throw an exception.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.between(Temporal, Temporal)` passing `this` as the first argument and the converted input temporal as the second argument.

This instance is immutable and unaffected by this method call.

Specified by:

`until` in interface `Temporal`

Parameters:

`endExclusive - the end date, exclusive, which is converted to a ChronoLocalDate in the same chronology, not null`

`unit - the unit to measure the amount in, not null`

Returns:

`the amount of time between this date and the end date`

Throws:

`DateTimeException - if the amount cannot be calculated, or the end temporal cannot be converted to a ChronoLocalDate`

`UnsupportedTemporalTypeException - if the unit is not supported`

`ArithmeticException - if numeric overflow occurs`

until

`ChronoPeriod until(ChronoLocalDate endDateExclusive)`

Calculates the period between this date and another date as a `ChronoPeriod`.

This calculates the period between two dates. All supplied chronologies calculate the period using years, months and days, however the `ChronoPeriod` API allows the period to be represented using other units.

The start and end points are `this` and the specified date. The result will be negative if the end is before the start. The negative sign will be the same in each of year, month and day.

The calculation is performed using the chronology of this date. If necessary, the input date will be converted to match.

This instance is immutable and unaffected by this method call.

Parameters:

`endDateExclusive - the end date, exclusive, which may be in any chronology, not null`

Returns:

`the period between this date and the end date, not null`

Throws:

DateTimeException - if the period cannot be calculated

ArithmeticException - if numeric overflow occurs

format

```
default String format(DateTimeFormatter formatter)
```

Formats this date using the specified formatter.

This date will be passed to the formatter to produce a string.

The default implementation must behave as follows:

```
  return formatter.format(this);
```

Parameters:

formatter - the formatter to use, not null

Returns:

the formatted date string, not null

Throws:

DateTimeException - if an error occurs during printing

atTime

```
default ChronoLocalDateTime<?> atTime(LocalTime localTime)
```

Combines this date with a time to create a ChronoLocalDateTime.

This returns a ChronoLocalDateTime formed from this date at the specified time. All possible combinations of date and time are valid.

Parameters:

localTime - the local time to use, not null

Returns:

the local date-time formed from this date and the specified time, not null

toEpochDay

```
default long toEpochDay()
```

Converts this date to the Epoch Day.

The Epoch Day count is a simple incrementing count of days where day 0 is 1970-01-01 (ISO). This definition is the same for all chronologies, enabling conversion.

This default implementation queries the EPOCH_DAY field.

Returns:

the Epoch Day equivalent to this date

compareTo

```
default int compareTo(ChronoLocalDate other)
```

Compares this date to another date, including the chronology.

The comparison is based first on the underlying time-line date, then on the chronology. It is "consistent with equals", as defined by Comparable.

For example, the following is the comparator order:

1. 2012-12-03 (ISO)
2. 2012-12-04 (ISO)
3. 2555-12-04 (ThaiBuddhist)
4. 2012-12-05 (ISO)

Values #2 and #3 represent the same date on the time-line. When two values represent the same date, the chronology ID is compared to distinguish them. This step is needed to make the ordering "consistent with equals".

If all the date objects being compared are in the same chronology, then the additional chronology stage is not required and only the local date is used. To compare the dates of two TemporalAccessor instances, including dates in two different chronologies, use ChronoField.EPOCH_DAY as a comparator.

This default implementation performs the comparison defined above.

Specified by:

compareTo in interface Comparable<ChronoLocalDate>

Parameters:

other - the other date to compare to, not null

Returns:

the comparator value, negative if less, positive if greater

isAfter

```
default boolean isAfter(ChronoLocalDate other)
```

Checks if this date is after the specified date ignoring the chronology.

This method differs from the comparison in compareTo(java.time.chrono.ChronoLocalDate) in that it only compares the underlying date and not the chronology. This allows dates in different calendar systems to be compared based on the time-line position. This is equivalent to using `date1.toEpochDay() > date2.toEpochDay()`.

This default implementation performs the comparison based on the epoch-day.

Parameters:

`other - the other date to compare to, not null`

Returns:

`true if this is after the specified date`

isBefore

```
default boolean isBefore(ChronoLocalDate other)
```

Checks if this date is before the specified date ignoring the chronology.

This method differs from the comparison in compareTo(java.time.chrono.ChronoLocalDate) in that it only compares the underlying date and not the chronology. This allows dates in different calendar systems to be compared based on the time-line position. This is equivalent to using `date1.toEpochDay() < date2.toEpochDay()`.

This default implementation performs the comparison based on the epoch-day.

Parameters:

`other - the other date to compare to, not null`

Returns:

`true if this is before the specified date`

isEqual

```
default boolean isEqual(ChronoLocalDate other)
```

Checks if this date is equal to the specified date ignoring the chronology.

This method differs from the comparison in compareTo(java.time.chrono.ChronoLocalDate) in that it only compares the underlying date and not the chronology. This allows dates in different calendar systems to be compared based on the time-line position. This is equivalent to using `date1.toEpochDay() == date2.toEpochDay()`.

This default implementation performs the comparison based on the epoch-day.

Parameters:

`other - the other date to compare to, not null`

Returns:

`true if the underlying date is equal to the specified date`

equals

`boolean equals(Object obj)`

Checks if this date is equal to another date, including the chronology.

Compares this date with another ensuring that the date and chronology are the same.

To compare the dates of two `TemporalAccessor` instances, including dates in two different chronologies, use `ChronoField.EPOCH_DAY` as a comparator.

Overrides:

`equals in class Object`

Parameters:

`obj - the object to check, null returns false`

Returns:

`true if this is equal to the other date`

See Also:

`Object.hashCode(), HashMap`

hashCode

`int hashCode()`

A hash code for this date.

Overrides:

`hashCode in class Object`

Returns:

`a suitable hash code`

See Also:

`Object.equals(java.lang.Object), System.identityHashCode(java.lang.Object)`

toString

`String toString()`

Outputs this date as a `String`.

The output will include the full local date.

Overrides:

`toString` in class `Object`

Returns:

`the formatted date, not null`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.